JNCIA-Junos Practice Lab
Features Implemented

Progress & Feature Inventory vs. Requirements – Revision 1

Project location: `C:\Users\HP\jnciapracticelabs`
Date: October 28, 2026
Companion document: *Project Needs & Requirements*

1. Status at a glance

Status	Meaning
IMPLEMENTED	Feature code exists in the project today and is exercised by the running server / engine modules.
IN PROGRESS	Actively being built next; design is pinned by the Requirements document.
PLANNED	Required by R-register but construction not started yet.

2. Implemented features (by module)

2.1 Lab delivery – zero-dependency runtime IMPLEMENTED

- `server/server.js` — plain Node.js static server (no npm dependencies): MIME map (html/css/js/json/svg/png/ico/md/txt), URL decoding, query-string stripping, `/index.html` default route, **path-traversal protection** (all files served strictly from `public/`), 404/403 responses, configurable port (`PORT` env, default 3000), clean startup banner with the lab URL and Ctrl+C instructions.
- `package.json` — project identity (`jncia-junos-lab` v1.0.0), engines `node >= 14`, scripts `npm start` (server) and `npm test` (test runner).
- `START-WINDOWS.bat` / `START-MAC-LINUX.sh` — one-click launchers so the lab runs without any package installation (R-14).
- **Offline-first:** the entire lab is served from local files; browser UI needs no internet access.

2.2 Junos configuration database (config tree engine) IMPLEMENTED

`public/js/junos/db.js` implements the heart of real Junos configuration handling:

- In-memory **hierarchical configuration tree** exactly shaped like Junos config (statement names, value lists, ordered children) with identity containers (`unit`, `address`, `interface`, `route`, `area`, `neighbor`, `term`, `filter`, ...) and leaves that carry values (`host-name`, `description`, `next-hop`, `peer-as`, ...).
- Core mutation primitives: `set` (incl. bracketed multi-values like `members [ a b ]`), `delete` (statement or single value, auto-cleanup of empty leaves), `deactivate / activate` (typical Junos `inactive:` marker behavior), `annotate` (stores `/* comment */` annotations).
- **Brace-notation rendering** faithful to real Junos: correct 4-space indentation chains, `inactive: ge-0/0/2 { ... }` headings, embedded comments (`/* SPARE */ ge-0/0/2 { ... }`), quoted values when needed, semicolons and nested braces exactly as Junos prints them in `show` / `show configuration`.
- **Set-notation rendering** (candidate & committed, active-only option) for `| display set` output; proper quoting (`set system description "LINK TO R2"`).
- `show | compare` **diff engine** (`compareTrees`): real Junos-style output with `[edit ...]` headers and `+ / - / !` line markers, subtree additions/deletions, value-level additions/deletions, and deactivation shown as `!` lines. Includes tree-equality checks (with and without the inactive marker) as used by the engine.
- Data utilities used across the lab: tree cloning, subtree lookup by path.

2.3 Lab inventory & baseline configurations IMPLEMENTED

`public/js/junos/model.js` defines the complete practice topology and every device's **factory baseline Junos configuration** as Junos `set` statements:

- **R1 / R2 / R3** — MX240 routers, Junos `20.4R2-S2.1`, deterministic serials, configured with `system` (host-name, time-zone UTC, ssh v2/root-login deny/limit), interface addressing (transit 10.0.12.0/30, 10.0.23.0/30; lo0 1.1.1.1–3.3.3.3/32; fxp0 mgmt 192.168.1.x), disabled spare ports with `SPARE` descriptions, `routing-options` (router-id, static default on R1, static discard on R3), **OSPF area 0.0.0.0** with passive lo0, and **internal BGP group PEERS** (65000, loopback-based peers).
- **SW1** — authentic EX-series shape: `EX4300-24T`, 24 ports, `family ethernet-switching` access/uplink ports with `vlan members`, **irb.10 / irb.20 SVIs**, `vlans staff(10) / guest(20)` with `l3-interface`, `me0` management port, and `protocols rstp interface all`.

- **Topology array** (R1–R2, R2–R3 routed; R1–SW1 L2) and **Host-A/Host-B** VPCS endpoints (switch port + VLAN + IP) used by every downstream simulation.

2.4 Runtime state derivation (the "network" inside the lab) IMPLEMENTED

`public/js/junos/derive.js` turns committed configs + the topology into consistent, Junos-like device state used by every console command — the same way real routers derive operational state from configuration:

- **Interfaces inventory**: configured and unconfigured ports (spare `ge-0/0/x` ports generated for realism), deterministic `Interface index / SNMP ifIndex / Generation`, MACs built from a stable hash (`02:xx:xx:xx:xx:xx`), admin/link flags from `disable` statements and peer port state, `last-flapped` from boot times.
- **Subnets / units**: parsed `family inet / inet6 / ethernet-switching` addresses, VLAN membership lists, interface-mode (trunk/access), per-unit MTUs.
- **Route table model** with real Junos protocol tags and preferences: `Direct/0`, `Local/8`, `Static/5`, `OSPF/10`, including `next-hop` / `discard` / `reject` static modes, correct active/non-active (`*` best-path) selection, and route age provenance.
- **Layer-2:** VLAN bridging domains from switch config + L2 links; IRB link state tied to `l3-interface` references; host ports derived from VLAN membership; realistic **(EX) MAC address table** and **STP** (RSTP) bridge/role model.
- **OSPF:** adjacency detection restricted to real same-subnet, non-passive, up interfaces; neighbor tables (`Full` state, dead time, priority 128, neighbor addresses), **Router LSAs** (seq/age/cksum/len with per-router values) and per-multiaccess-net LSAs, plus **Dijkstra SPF** over the actual adjacencies to build OSPF routes with hop costs and validated next-hops.
- **ARP & LLDP:** routers learn neighbors' MACs on shared subnets (including SVI/uplink bridging through the switch), and LLDP neighbor lists are derived from physical links with real chassis IDs and port info.
- **BGP:** session state machine derived both ways (peer must be reciprocally configured): `Establ` with packet counters/uptime, `Active`, or `Idle` when prerequisites (peer config, AS, route, local-address) are missing — exactly the states `show bgp summary` would display.
- **Ping/traceroute resolution engine:** source-address selection (explicit `source` check, egress-interface choice), longest-prefix route matching, hop-by-hop path walk including no-route / discard / no-host / loop outcomes — this drives authentic `ping` / `traceroute` results.

2.5 Shared engine utilities IMPLEMENTED

`public/js/junos/util.js` (namespace `JJ`): IPv4 integer math (`ipToInt/intToIp`), network math (`netOf`, `inNet`, `cidrHost/cidrBroadcast`, subnet membership), deterministic MAC generator, time rendering in Junos formats (UTC stamps, `Oct 28 06:03:12` style, uptime strings like `5 days, 03:47`, ages like `00:12:43`), port-number-safe token comparators for interface name/number ordering, and column padding used by every table builder (perfect column alignment like Junos output).

3. Preserved exactness guarantees (already enforced by the engine)

- Protocol tags and preferences appear exactly as Junos prints them: `[Static/5]`, `[Local/8]`, `[OSPF/10]`, `[Direct/0]`.
- Active-route marking, `+ = Active Route, - = Last Active, * = Both` legend support, and OSPF event ages/seq numbers derive from stable device state (uptime, commit count).
- Peer states render with the exact Junos spelling: `Establ`, `Active`, `Idle`; OSPF neighbor `Dead time` in `00:00:35` form; priorities `128`.
- Loopback routes show as `/32` per-interface `Local via` style entries.
- `show | compare` headers use `[edit ...]` scopes and per-line `+/-/!` markers exactly like Junos diffs.

4. Work in progress — next delivery items

Status	Item	Fulfills
IN PROGRESS	**Junos CLI machine** (`public/js/junos/machine.js`): per-device runtime (committed config + candidate + revisions + commit log), session lifecycle (boot messages, `login:` , root shell `root@R1:~ #` , `cli`), operational-mode grammar with exact Junos `show/ping/traceroute/monitor/request/.../` outputs from Section 4 of the Requirements doc, config-mode grammar (`set/delete/.../commit/rollback`) with schema validation, completions (`?` /Tab), pipes, error strings and `---(more)---` pagination semantics.	R-01–R-07, R-15, R-16
IN PROGRESS	**Terminal widget** (`public/js/ui/console.js`): black-on-white GNS3 telnet-style rendering, char-level editing (arrows, backspace, insertion), history, `?` /Tab behavior identical to Junos, pager interaction (space / b / q / ?), Ctrl+C abort.	R-04, R-10
IN PROGRESS	**GNS3-like shell** (`public/js/ui/app.js` + `public/css/app.css` + `public/index.html`): menu bar, GNS3 toolbar, dark topology canvas with draggable router/switch/PC icons, link lines + optional subnet labels, node context menus (Start/Stop/Reload/Console), console windows titled like `R1 - telnet://127.0.0.1:5000` , Console Manager, right dock Topology Summary & Servers Summary, status bar, zoom/pan/grid/view toggles, GNS3-like project save/open (localStorage persistence), Admin dialogs (About).	R-09–R-13
PLANNED	**VPCS host console** (`public/js/ui/vpcs.js`): `Host-A>` prompt and VPCS-style `ping/trace/show/show ip/show arp/clear/help/quit` output for devices other than Junos.	R-08
PLANNED	**Test suite** (`tests/run-tests.js`): engine + CLI-machine checks (config render, commit/rollback, OSPF/ARP/route outputs, ping scenarios, exact error strings).	R-14 (A10)

5. How to run what exists today

```
cd C:\Users\HP\jnciapracticelabs
START-WINDOWS.bat         (or:  npm start  /  node server\server.js)
# then open http://localhost:3000
```

The server and engine modules run today; the browser console experience arrives with the CLI machine + UI items listed in Section 4, built strictly to the exact-output references pinned in the Requirements document.

6. Requirement traceability snapshot

ID (Needs doc)	State today
R-01 / R-02 / R-03 (Junos-exact behavior and output)	Engine substrate complete (models, config DB, exact rendering primitives, runtime state); full CLI surface being built now in `machine.js`.
R-04..R-07 (CLI behavior, config workflow, EX output, console flow)	Rendering/diff/compare layers ready; grammar + session machine in progress; SW1 EX-family tables derive already (VLANs, MAC table, STP).
R-08 (VPCS hosts)	Planned (the general machine infrastructure is needed first).
R-09..R-11 (GNS3 UI + lifecycle)	UI shell in progress; lifecycle actions will operate on the runtime registry.
R-12 (topology inventory)	**IMPLEMENTED** in `model.js`; canvas depiction follows with the UI.
R-13 (persistence)	Design ready (runtime serialization format in engine); persistence wiring ships with `machine.js` project layer + UI project save.
R-14 (offline, zero deps)	**IMPLEMENTED**.
R-15 / R-16 (interpretability, administration commands)	Resolution engine ready (ping/traceroute); admin commands arrive with the CLI grammar.